

in this process; hence, installing a preconfigured package is recommended for Windows. Preconfigured packages are software installers that do the steps listed above, for you. All you need to do is to install them and you are ready to go!

The one that is most widely used is XAMPP, which I recommend because it is popular—it means that if you encounter a problem, there will be enough help available on the Web. The second (and quite important) parameter is path names. Where you install software in Windows is an important factor, more so when you are running software originally built for UNIX-like platforms (or let's just say 'Linux' instead). Windows and Linux deal with file system paths in different ways, which can be annoying. The installation paths for the components (Apache, PHP and MySQL) are conducive to the Windows environment. The third reason for choosing XAMPP is that it comes with a simple service controller. So you can launch Apache and MySQL when you need them and shut them down when you don't. Fourth, XAMPP also comes with a nifty FTP client, which can be pretty useful when you need to upload your work to a server. The last reason is that XAMPP also comes with other necessary tools, which allow you to get started real fast; for example, the MySQL root password is set to blank, PhpMyAdmin (a Web or browser-based tool to manage MySQL databases) is already installed and there is a page showing 'phpinfo()'. Please note that the setup created by XAMPP is nowhere close to being secure enough for use on a production server.

Moving on with PHP on Windows

Installing PHP on Windows is only the beginning, to be followed by installing extensions, which is much simpler. Most of the necessary extensions are already installed. All you have to do is to enable them in the *php.ini* file and they will start working. So connecting to your databases or encrypting your data using PHP isn't a problem.

But as you move forward, it is important to keep in mind that PHP's primary target is not Windows. PHP is UNIX-world software that 'also' runs on Windows. One of the biggest problems for new programmers using PHP on Windows is the path names. If you open a file and you copy-paste the path from your Windows Explorer's address bar into your PHP script, it may not work—one of the most common problems.

File paths

On all UNIX-like systems, the directory separator is the '/' character. On Windows, it is '\'. At many points, this might cause some confusion upfront. While '/' might work at times when specifying the path in PHP, it should be avoided. The worst thing you can do with paths is to mix the directory separators in the same path, which will not work. PHP recommends that you use '/' as the directory separator on

Windows as well. So `C:\xampp\htdocs\project` should be written as `C:/xampp/htdocs/project`.

Line endings and IDEs

Development involves using other people's work—a library, a patch or maybe even a complete project. Linux uses the '\n' character for newline while Windows uses '\r\n'. This means that if you are using a piece of code which was originally written for Linux, it may not look pretty on Windows; at least, not natively. If you have been using a primitive text editor (such as Notepad) for coding, make sure that you get yourself an IDE or at least a decent text editor with more developer-friendly features. Notepad++ and Sublime Text are very developer-friendly text editors. Eclipse for PHP is a good free IDE while PHPStorm is by far the best IDE you can get for PHP, with many of its features currently not seen in most other IDEs. If you like Visual Studio, there is a plugin by Microsoft that allows you to write PHP with Visual Studio as well. Get yourself a decent IDE/text editor. These not only fix the viewing problems associated with line-endings but also come with some goodies that help you work much faster.

Servers

PHP is a scripting language for the Web. You do need a Web server to make it function and there are a number of them. On Windows, you have at least one extra option compared to other platforms—IIS. This stands for Internet Information Services and is the Web server developed by Microsoft itself. There are two things about it that need to be remembered:

1. It is switched off, by default.
2. If you are not going to work on C# or other .NET languages for Web development on Windows, it is better to use Apache rather than IIS. It does exactly what Apache does—serve Web pages. However, IIS supports Microsoft technologies, which Apache does not do too well. However, when it comes to PHP, the amount of help you get online with Apache is much more than what you get for IIS. The reason being that Apache works on Linux too, and with the exception of file paths, most solutions that work on Linux work on Windows as well.

And like I have already said, it is better to go for packaged AMP installations such as XAMPP which come with Apache. Enabling IIS would create a conflict with them.

Apache

Apache is the most famous Web server there is. What separates Apache from the rest is the amount of customisation that you can do. Having a threaded server or a non-threaded one, support for a very large number of languages as modules, handling multiple websites, managing permissions, customisable security levels are some of the most attractive functions that one can tweak the server to deliver. All of this means that there are chances that you can get one of those